

Module 7 Week B — Core Skills Drill: Pre-Trained Pipelines & Metrics

This drill is due the evening of the concept session (Sunday). It moves you from "I read about pipelines, exact-match (EM) and token-F1, and Recall-Oriented Understudy for Gisting Evaluation (ROUGE)" to "I can construct a Question Answering (QA) pipeline, a summarization pipeline, and the two evaluation functions, and run them on a fixture without supervision." You will not evaluate real datasets in the drill — that is tomorrow's lab and Wednesday's integration. The drill targets the mechanical pieces both will assume.

Due: Tonight, after the concept session. Resubmissions are accepted through Saturday of the assignment week.

Setup

1. **Accept the assignment**
2. **Clone your repo:**

```
git clone <your-repo-url>
cd <your-repo-name>
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

This installs `transformers`, `torch` (CPU build), `pandas`, `numpy`, and `rouge-score` (new in Module 7 Week B). The first four are already installed if you completed Week A; `rouge-score` is the only new package.

4. **Create a working branch:**

```
git checkout -b drill-7b-pretrained-pipelines
```

All your work goes in `drill.py`. The starter file contains function signatures and docstrings — implement each function as described below.

Note on first run: Tasks 1 and 4 construct Hugging Face pipelines, which download model weights on first use (~250 MB each). Plan ~3–5 minutes for the first run; subsequent runs use cached weights.

Task 1: Build a QA Pipeline and Answer One Question

Implement `build_qa_pipeline(model_name)` and `answer_one(qa, question, context)`:

- `build_qa_pipeline` constructs a Hugging Face `pipeline("question-answering", model=model_name)` and returns it.
- `answer_one` calls the pipeline with the given question and context, and returns the full result dictionary (not just the answer string).

```
def build_qa_pipeline(model_name: str):
    """
    Construct a Hugging Face question-answering pipeline.

    Returns the pipeline object (callable).
    """
    # TODO: import pipeline from transformers
    # TODO: return pipeline("question-answering", model=model_name)

def answer_one(qa, question: str, context: str) -> dict:
    """
    Run the QA pipeline on one (question, context) pair.

    Returns the pipeline output dictionary with keys:
    "answer", "score", "start", "end".
    """
    # TODO: call qa(question=..., context=...) and return the result
```

Why this matters: The lab evaluates the pipeline over many examples. The drill verifies you can construct it and parse one output before scaling up.

Task 2: Implement Answer Normalization and Exact-Match

Implement `normalize_answer(s)` and `exact_match(pred, gold)`:

- `normalize_answer` applies the standard SQuAD normalization: lowercase, strip articles (`a`, `an`, `the` as standalone words), strip punctuation, collapse whitespace.
- `exact_match` returns `1` if `normalize_answer(pred) == normalize_answer(gold)`, else `0`.

```
import re
import string

def normalize_answer(s: str) -> str:
    """
    Apply SQuAD-style normalization:
    - lowercase
    - strip articles (standalone "a", "an", "the")
    - strip punctuation
    - collapse whitespace
    """
    # TODO: lowercase
    # TODO: strip articles using a word-boundary regex (so "the" in "thereby" su
    # TODO: remove all string.punctuation characters
    # TODO: collapse whitespace and strip

def exact_match(pred: str, gold: str) -> int:
    """
    Return 1 if normalized prediction equals normalized gold, else 0.
    """
    # TODO: normalize both, compare, return int
```

Common pitfall: stripping articles without a word boundary mangles other words. Use `re.sub(r"\b(a|an|the)\b", " ", s)` — the `\b` anchors prevent matching inside other words.

Task 3: Implement Token-F1

Implement `token_f1(pred, gold)`:

- Normalize both strings with `normalize_answer`.
- Split each on whitespace into a list of tokens.
- Compute `precision = overlap / |predicted|`, `recall = overlap / |gold|`, `F1` = harmonic mean.
- Return `0.0` if either side is empty (do not return `NaN`).
- Return `1.0` if both are empty (this is the convention for SQuAD v2.0 no-answer; you should still implement it correctly).

```
def token_f1(pred: str, gold: str) -> float:
    """
    Compute token-F1 between prediction and gold after normalization.

    Returns a float in [0.0, 1.0].
    """
    # TODO: normalize both, split into token lists
    # TODO: handle empty cases (both empty -> 1.0; one empty -> 0.0)
    # TODO: compute overlap as the intersection size of the token bags
    # TODO: compute precision, recall, return harmonic mean
```

Common pitfall: dividing by zero when predicted is empty produces a `NaN` that silently propagates through aggregation. Handle the empty case explicitly.

Task 4: Build a Summarization Pipeline and Summarize One Document

Implement `build_summarizer(model_name)` and `summarize_one(summ, text, max_length, min_length)`:

- `build_summarizer` constructs a `pipeline("summarization", model=model_name)`.
- `summarize_one` calls the pipeline with the text and length constraints, with `do_sample=False` and `num_beams=4`. Return the `summary_text` field of the first output (the pipeline returns a list).

```
def build_summarizer(model_name: str):
    """
    Construct a Hugging Face summarization pipeline.

    Returns the pipeline object (callable).
    """
    # TODO: return pipeline("summarization", model=model_name)

def summarize_one(summ, text: str, max_length: int, min_length: int) -> str:
    """
    Run the summarization pipeline on one document.

    Returns the summary string (not the pipeline's wrapper list/dict).
    """
    # TODO: call summ(text, max_length=..., min_length=..., do_sample=False, num_
    # TODO: extract and return the "summary_text" field of the first element
```

Common pitfall: the pipeline returns `[{"summary_text": "..."}]` — a list of length 1. Forgetting to index `[0]["summary_text"]` returns the list, which fails downstream type checks.

Task 5: Compute ROUGE for One (Predicted, Reference) Pair

Implement `compute_rouge(pred, ref)`:

- Construct a `RougeScorer([{"rouge1", "rouge2", "rougeL"}, use_stemmer=True)]`.
- Call `scorer.score(reference, predicted)` — **reference first, then predicted**.
- Extract the F1 measure from each variant's result.
- Return a dictionary `{"rouge1": float, "rouge2": float, "rougeL": float}`.

```
from rouge_score.rouge_scorer import RougeScorer

def compute_rouge(pred: str, ref: str) -> dict:
    """
    Compute ROUGE-1, ROUGE-2, and ROUGE-L F1 between predicted and reference.

    Returns a dict with keys "rouge1", "rouge2", "rougeL", each a float in [0.0,
    """
    # TODO: construct a RougeScorer with the three metric names and stemmer enabl
    # TODO: call scorer.score(ref, pred) — note the argument order
    # TODO: extract .fmeasure from each result and return as a dict
```

Common pitfall: the argument order of `scorer.score` is (target, prediction) — the **reference / target first**. Reversing the order does not raise an error (the result is symmetric for F1 in a single-reference setting), but mixing this up makes per-summary debugging confusing.

Submitting

When all tasks are implemented and your local tests pass:

1. **Stage and commit your changes:**

```
git add drill.py
git commit -m "Complete drill 7B"
```

2. **Push your branch:**

```
git push -u origin drill-7b-pretrained-pipelines
```

3. **Open a Pull Request** from your branch into `main`. The autograder runs automatically; check the green check before submitting.

Your PR description must include:

- One-line summary of what each task does.
- Which task you found most challenging and why (1–2 sentences).
- Paste your PR URL into TalentLMS → Module 7 → Core Skills Drill 7B to submit this assignment.

What's Next

- Tomorrow (Day 2, Monday):** Applied Lab — pre-trained QA on a curated tech-news QA subset, full evaluation harness, failure-mode analysis.
- Wednesday (Day 4):** Integration Task — pre-trained summarization on the tech news corpus + the integrated evaluation report (the M7 deliverable).

